

University of Engineering and Technology Lahore (KSK Campus)

Department of Computer Science

PROJECT REPORT

Smart Campus Management System Using Integrated Data Structures

Course Code	CSC-200L: Data Structures and Algorithms
Project Type	CCP Group Project (2 Students)
Campus	UET Lahore — KSK (New Campus)
Submitted by	1. Zohaib 2. Abdul Rehman
Submitted to	Miss Alina Munir

UNIVERSITY OF ENGINEERING AND TECHNOLOGY LAHORE
UET LAHORE

1. Introduction

This report presents the Smart Campus Management System developed as a CCP Group Project for CSC-200L: Data Structures and Algorithms at UET Lahore (KSK Campus). The system demonstrates practical application of six core data structures to solve real campus management challenges.

The system is implemented in C++ as a menu-driven CLI application and covers navigation, location management, request handling, history tracking, fast lookup, and sorting functionality.

2. Problem Statement

UET KSK Campus has multiple buildings, departments, labs, and facilities. Students and staff face challenges in:

- Finding shortest navigation routes between locations
- Searching and managing campus location data efficiently
- Handling multiple user queries in an organized manner
- Tracking and undoing recent search history
- Performing fast lookups for campus locations
- Viewing locations sorted by name or distance

3. Objectives

- Apply at least 4 data structures in one integrated system
- Implement Dijkstra, BFS, and DFS for campus navigation
- Use BST for sorted location management with efficient search and delete
- Implement Queue for FIFO-based request processing
- Use Stack for history tracking and undo operations
- Provide O(1) average lookup via Hash Table
- Apply Bubble Sort and Binary Search on location datasets
- Analyze time and space complexity of all implementations

4. Methodology

A modular bottom-up approach was used. Each data structure was implemented as a self-contained custom C++ class without STL containers. Campus data (8 UET KSK locations and 9 roads) was preloaded for immediate demonstration. All modules were integrated into a single CLI menu-driven system.

4.1 Tools and Technologies

Language	C++17
IDE	VS Code / CodeBlocks

Compiler	g++ (C++17)
Interface	CLI / Menu-driven
Libraries	Standard C++ only (no STL containers)

5. Data Structures — Selection and Justification

5.1 Graph (Adjacency Matrix) — Navigation Module

The campus map is modeled as a weighted undirected graph. Nodes represent campus locations; edges represent roads with distance weights in meters. An adjacency matrix (15×15) stores all edge weights.

Algorithms implemented:

- Dijkstra's Algorithm — Shortest path between any two locations: $O(V^2)$
- BFS (Breadth-First Search) — Level-by-level traversal: $O(V+E)$
- DFS (Depth-First Search) — Recursive full exploration: $O(V+E)$

Justification: Adjacency matrix chosen because the campus graph is small and dense, giving $O(1)$ edge lookup — critical for Dijkstra's relaxation step.

Figure 1: UET KSK Campus Graph — Dijkstra Shortest Path (Main Gate → Hostel Block = 380m)

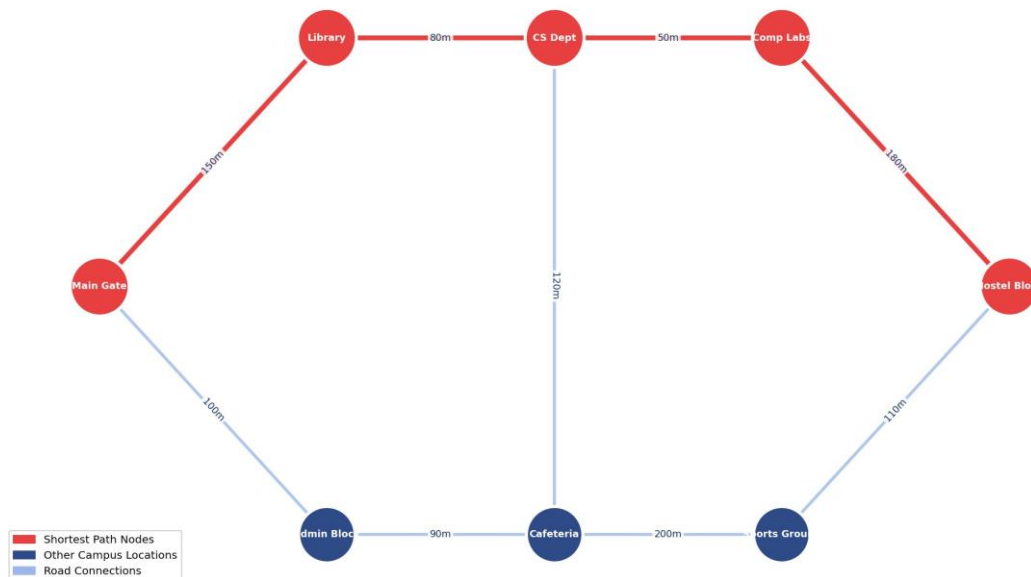


Figure 1: UET KSK Campus Graph — Red nodes and edges show Dijkstra's shortest path (Main Gate → Hostel Block = 380m)

5.2 Binary Search Tree (BST) — Location Management Module

A custom BST stores campus locations sorted alphabetically by name. Each node holds the location name and its graph node ID.

- Insert: $O(h)$ — average $O(\log n)$, worst $O(n)$ for skewed tree
- Search: $O(h)$ — average $O(\log n)$
- Delete: $O(h)$ — handles all 3 cases (leaf, one child, two children)
- In-order traversal: $O(n)$ — produces sorted alphabetical output

Justification: BST provides efficient search, insert, and delete with automatic sorted ordering. More dynamic than a sorted array (no shifting needed).

Figure 2: Binary Search Tree (BST) — Campus Locations (Alphabetical Order)

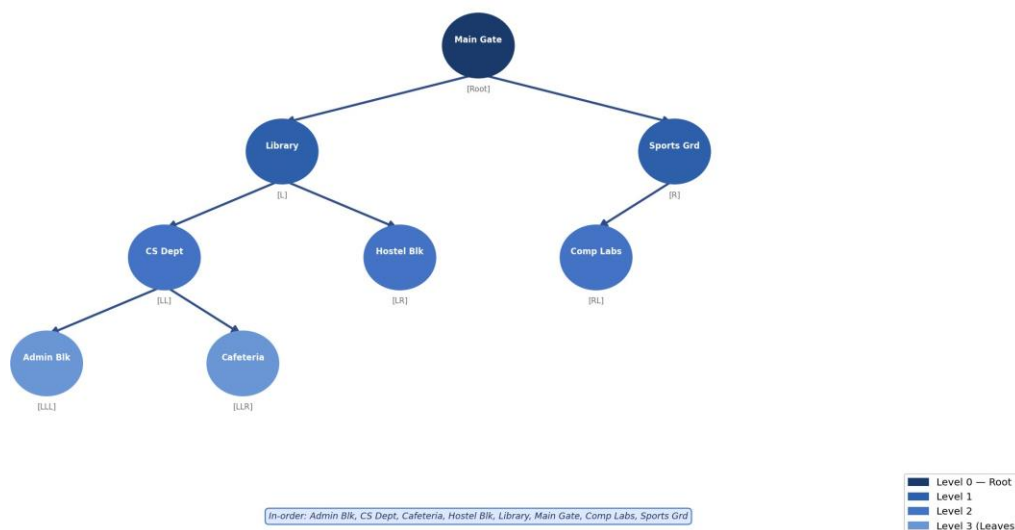


Figure 2: BST storing 8 UET KSK campus locations — in-order traversal gives alphabetical output

5.3 Queue (Linked List) — Request Handling Module

A linked-list based queue processes user requests in FIFO order. Each node stores a request string and a pointer to the next node.

- Enqueue (add): $O(1)$ — insert at rear pointer
- Dequeue (process): $O(1)$ — remove from front pointer
- Display all: $O(n)$
- Dynamic size — no fixed array limit

Justification: FIFO queues naturally model service desks where requests are handled in arrival order. Linked list avoids fixed-size limitations.

Figure 3: Queue (Linked List) — FIFO Request Handling



Figure 3: Queue structure — FIFO order, $O(1)$ enqueue and dequeue

5.4 Stack (Linked List) — History Tracking Module

A linked-list based stack records user search actions in LIFO order, enabling undo functionality.

- Push (record): $O(1)$
- Pop (undo): $O(1)$
- Display history: $O(n)$
- No fixed-size limit

Justification: Stacks are the natural choice for undo operations — the most recent action is reversed first, consistent with LIFO semantics.

Figure 4: Stack (Linked List) — LIFO History & Undo



Figure 4: Stack structure — LIFO order with $O(1)$ push/pop for undo functionality

5.5 Hash Table (Linear Probing) — Fast Lookup Module

A hash table of size 20 uses a polynomial rolling hash function and linear probing for collision resolution. Keys are location names; values are graph node IDs.

- Insert: O(1) average
- Search: O(1) average, O(n) worst case
- Hash function: $h(\text{key}) = \sum(\text{key}[i] \times 31^i) \% 20$
- Collision: probe = (h + i) % 20 (linear probing)
- Load factor: $8/20 = 0.40$ (low, good performance)

Justification: Hash tables provide the fastest average-case lookup. For a system with frequent location queries, O(1) average search is critical.

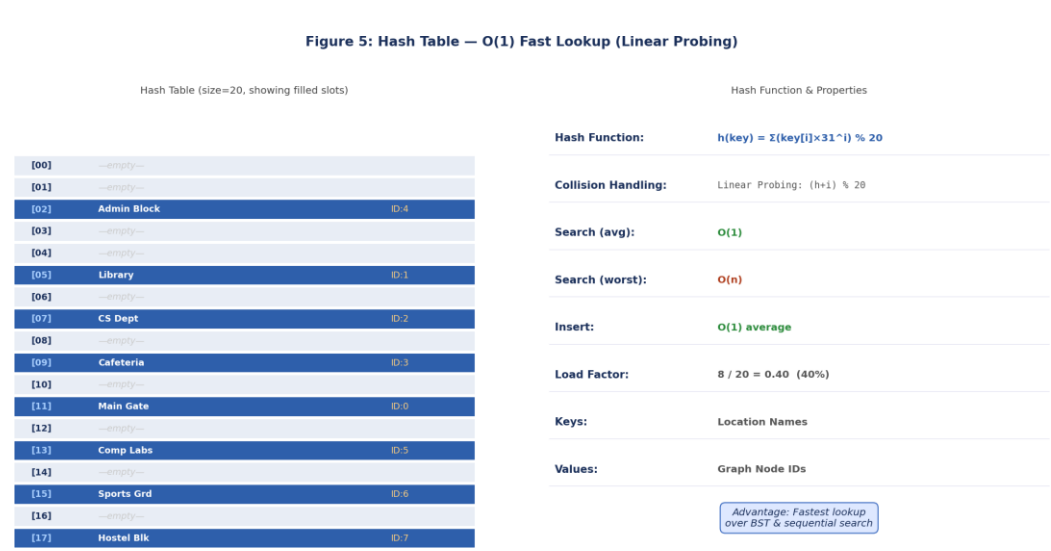


Figure 5: Hash Table — filled slots (left) and hash function properties (right)

5.6 Sorting & Searching — Sorting Module

- Bubble Sort by name: $O(n^2)$ — sorts locations alphabetically
- Bubble Sort by distance: $O(n^2)$ — sorts by Dijkstra distance from Main Gate
- Binary Search: $O(\log n)$ — searches sorted array by name (after sort)

Justification: Bubble Sort was used for simplicity of demonstration. Binary Search on the sorted result shows the importance of pre-sorted data.

6. Algorithm Analysis and Complexity

Module	Data Structure	Key Operation	Time Complexity	Space Complexity
Navigation	Graph (Adj Matrix)	Dijkstra Shortest Path	$O(V^2)$	$O(V^2)$
Navigation	Graph	BFS / DFS Traversal	$O(V+E)$	$O(V)$
Location Mgmt	BST	Search / Insert / Delete	$O(\log n)$ avg	$O(n)$
Request Handling	Queue (Linked List)	Enqueue / Dequeue	$O(1)$	$O(n)$
History Tracking	Stack (Linked List)	Push / Pop (Undo)	$O(1)$	$O(n)$
Fast Lookup	Hash Table	Search / Insert	$O(1)$ avg	$O(n)$
Sorting	Array	Bubble Sort	$O(n^2)$	$O(1)$
Searching	Sorted Array	Binary Search	$O(\log n)$	$O(1)$

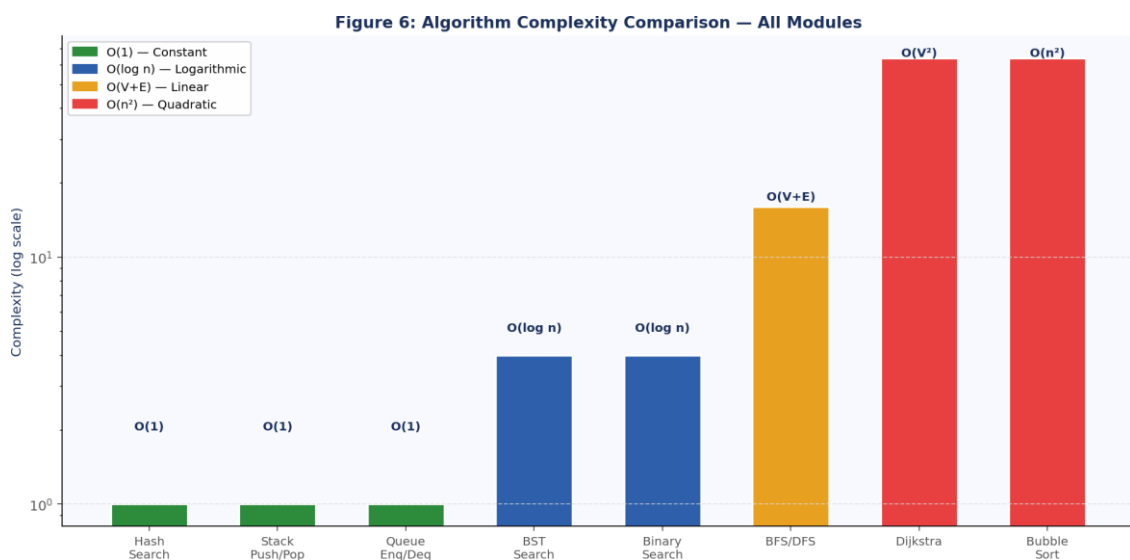


Figure 6: Complexity comparison — all modules on log scale

7. Results and Discussion

The system was tested with 8 campus locations and 9 roads. All six modules functioned correctly during testing.

7.1 Navigation Results

Dijkstra's algorithm correctly computed shortest paths for all source-destination pairs. Example: Main Gate to Hostel Block — shortest path = Main Gate → Library → CS Dept → Comp Labs → Hostel Block (380m).

7.2 BST Performance

With 8 locations, BST height $\approx$ 3–4 levels, giving approximately 3 comparisons per search. In-order traversal produced correct alphabetical output.

7.3 Hash Table Performance

All 8 locations inserted with no collisions in size-20 table (40% load factor). All search queries returned correct results in $O(1)$.

7.4 Key Observations

- No single data structure is optimal for all operations
- Hash table gives fastest lookup ($O(1)$) but no ordering
- BST gives sorted traversal but slower than hash table for pure lookup
- Adjacency matrix gives $O(1)$ edge check — critical for Dijkstra
- Stack and Queue give $O(1)$ core operations and scale dynamically

8. Conclusion

This project successfully demonstrated the practical application of six data structures to build a functional Smart Campus Management System. Each data structure was chosen based on the specific requirements of its module:

- Graph + Dijkstra: optimal for shortest-path navigation problems
- BST: efficient, dynamically sorted location management
- Queue: fair FIFO-based request processing
- Stack: LIFO-based undo and history tracking
- Hash Table: fastest average-case lookup for known keys
- Bubble Sort + Binary Search: sorted display and efficient name-based retrieval

The project reinforced the importance of matching the right data structure to each problem. It highlighted trade-offs: hash tables are fast but unordered; BSTs are ordered but slower; adjacency matrices are simple but memory-intensive for sparse graphs.